BootBarn.com — HUMAN Security (PerimeterX) blocker assessment (2026-10-31)

Subject: the bot blockers on `bootbarn.com` (Salesforce Commerce Cloud)
Method: public-site analysis + runtime observation. All 3rd-party telemetry was **observed or fulfilled locally**; nothing forwarded, no customer/prod traffic touched.
Result: BootBarn runs **HUMAN Security** (formerly White Ops / PerimeterX) as its main bot blocker (via the classic `PX-Show` entry point), alongside **Signifyd** (device-intel, sibling repo target) + OneTrust consent + Yottaa + Cimulate copilot. We mapped the full HUMAN client stack, decoded the string table, read the plaintext collector telemetry at the JSON boundary, and identified what the server uses to decide "*Access to this page has been denied — we believe you are using automation tools.*"

---

1. The blocker stack (what actually loads)

Layer	Product	Evidence
HUMAN Security (PerimeterX)	bot mitigation / risk scoring	`PX-Show` redirect, `_px*` globals, `client.px-cloud.net/PXJeSgWPZK/main.min.js`, `captcha.px-cdn.net`
Signifyd	device intelligence	`cdn-scripts.signifyd.com/api/script-tag.js`
OneTrust	cookie consent	`cdn.cookielaw.org`
Yottaa	performance/CDN	`rapid-cdn.yottaa.com/rapid/lib/…`
Cimulate	search copilot	`cdn.search.cimulate.ai/copilot-widget/…`

The HUMAN integration is a **block-page deployment** (not just scoring): a suspicious session gets `PX-Show` → a page that says "*Please verify you are a human*" with the classic denial text.

2. The HUMAN client surface (8 hosts observed)

Host	Role
`client.px-cloud.net`	serves `main.min.js` (core client, 249 KB)
`captcha.px-cdn.net`	serves `captcha.js` (610 KB challenge, `hc_embedded`)
`collector-pxjesgwpzk.px-cloud.net`	**telemetry** — `/api/v2/collector` + `/assets/js/bundle` (behavior)
`collector-pxjesgwpzk.px-client.net`	fallback collector (`/b/g`)
`ift.px-cloud.net`	`/ns` — fingerprint/device server
`ri.px-cloud.net`	`/index.js` — risk-injection
`fst-ec.px-cloud.net`	session/timing endpoint
`dvp.px-cloud.net`	`/v` — **device verification protocol** (encrypted blob)

Cookies: `_pxhd`, `_pxvid`, `_px3`, `pxcts` (the PX identity/risk cookie set).

3. The string table — decoded (525 plaintext strings)

`main.min.js` obfuscates strings with a **custom base91** scheme: 645 entries in a string array, decoded by a bit-packing base91 decoder using a 91-char permuted alphabet, then UTF-8. (Two alphabets are present — the active one is the IIFE-local `{u&Yj<…>6>UX`; the top-level `G]KLY+…` is a decoy.)

Decoded signal names include `sessionStorage`, `error`, and the full set of collector/API paths (`/api/v2/collector`, `clientError`, `captcha`).

4. The collector telemetry — readable plaintext (the money shot)

Hooking `JSON.stringify` at the boundary (before PX encodes it) reveals the exact JSON the client ships. It is then XOR/encoded into the `payload=<hex-or-b64>` form seen on the wire — so the wire format is opaque but the **pre-encoding plaintext is fully readable** (same "read at the boundary" playbook as every vendor in this project).

Navigation event (913 B):


```
{
  "commonDimensions": { "optState": "active" },
  "connection": { "effectiveType": "4g", "rtt": 0 },
  "fullPageURL": "https://www.bootbarn.com/on/demandware.store/…/PX-Show?url=…",
  "metaEvents": [ { "name": "initial-session", "metadata": { "sessionId": "4ae95d78-…", "isConfigCached":
  "releaseDate": "v9.0.4-2026-09-29T21:22Z",
  "requestType": "navigation",
  "siteKey": "AlF2x2KZXmxk3g",
  "userAgent": "Mozilla/5.0 … HeadlessChrome/148…",
  "uuid": "7092a0f2-…",
  "sessionTracking": true, "navType": "hard",
  "navigationIdentifier": "hard-a05706df-…"
}
```


Session/visitor meta (215 B): `visitorId`, `sessionId`, page/session counts, `bandwidth`.

Behavior bundle (3,592 B): adds DOM/resource metrics — `domElementCount`, `entryMap` (script/link/beacon/img counts + bytes), `isNavigationSupported`, `isPaintTimingSupported`, `isResourceTimingSupported`, `lastInteraction`, `maxScrollDepth`, `pagePrefetched`, `pageTitle:"Access to this page has been denied."`, `sequencing` (delayed objects, block sequencing).

Session heartbeat (328 B): `{v:<uuid>, a:"PXJeSgWPZK", l:<page>, i:77, d:<ts>, h:10000000, p:"hc_embedded"}` — the risk-injection handshake.

5. The embedded WebGL fingerprint shader

`main.min.js` embeds a **base64 WebGL fragment shader** (decoded, 572 chars):

```
precision highp float;
uniform vec2 res;
const int MAX_LOOPS = 152; const int MINEXP = 120;
void main(){
  float y = (gl_FragCoord.y / res.y) * 32.0;
  float x = 1.0 - (gl_FragCoord.x / res.x);
  float fade = fract(pow(2.0, floor(y)) + x);
  ... for (int i=0;i<MAX_LOOPS;i++) if (i < row) xx *= 0.5; ...
}
```

This is HUMAN's **float-precision / shader-math probe** — it renders a hidden scene whose output depends on the GPU/WebGL implementation (driver, precision, emulation). Software renderers (SwiftShader/headless GL) produce a different result than real GPUs. It's one of the classic "is this a real browser/GPU" signals, and headless — where `UNMASKED_RENDERER` reports SwiftShader — is exactly what this catches.

6. Why we got blocked (the decision inputs)

The block text: "*Access to this page has been denied because we believe you are using automation tools... Javascript is disabled, ...*". Observed inputs the server had:

- `userAgent: …HeadlessChrome/148…` — the UA alone is a strong signal.
- WebGL renderer = SwiftShader (software) via the shader probe.
- `rtt: 0`, synthetic `effectiveType` — a real network stack differs.
- Automation-flag launch (Playwright/headless) — same launch-signature issue as the hCaptcha work.
- Session fingerprints across collector + dvp + ift correlate server-side.

6b. Clean-launch result (the decision flips)

Same as hCaptcha: launching **real Chrome manually** (fresh profile, no automation flags) changes the verdict. Observed:

	Playwright headless	Clean-launch real Chrome
Page	"Access to this page has been denied" block	**Full storefront** ("Cowboy Boots...", 535 links, 71 imgs)
HUMAN traffic	client → ift → collector → dvp → captcha.js	client → ift → collector (same flow, **no captcha**)
PX cookies	`_pxhd`, `_pxvid`, `_px3`, `pxcts`	**same set**
Signifyd	loads	loads (`window.signifyd` true)
`navigator.webdriver`	true	false

Conclusion: the HUMAN server's "automation" verdict is driven by the **launch/process signature** (automation flags, headless UA, software GL), *not* by anything the client module reveals differently. The signal flow is identical; the score changes because the session *is* different at the transport/launch layer. Same ladder as hCaptcha: a clean launch clears the first gate; the real defense is the server-side scoring + (when triggered) the captcha challenge.

7. How it compares (cross-vendor)

	HUMAN (BootBarn)	hCaptcha	Signifyd (sibling)
Client-readable plaintext?	**Yes** (JSON boundary)	No (context-gated WASM)	Yes (SubtleCrypto boundary)
Client-mintable token?	**No** — server scores + issues captcha/block	No	Partially (device_id)
Strongest signal	WebGL shader + launch signature + behavioral bundle	server gate + ML challenge	ECDH+AES-GCM (unbreakable)
Crypto-RE effort	**~4** (XOR wire + base91 strings — trivial)	n/a (couldn't init)	**2** (unbreakable, read at boundary)
Resistance to determined RE	**~65**	~85	~35

8. Artifacts

File	What
`px-main.js`	HUMAN core client (249 KB, from CDN)
`px-captcha.js`	HUMAN captcha challenge (610 KB)
`scripts/bb-probe.mjs`	load page, capture requests + globals
`scripts/bb-px-posts.mjs`	capture collector/dvp POST bodies
`scripts/bb-px-boundary.mjs`	capture plaintext JSON at the stringify boundary
`scripts/bb-px-trace.mjs`	iw() decoder patch
`scripts/bb-final.mjs`	block page + host matrix + cookies + screenshot
`artifacts/px-posts.json`	the wire POST bodies (XOR-encoded)
`artifacts/boundary.json`	the plaintext telemetry (decoded)
`artifacts/px-strings.json`	525 decoded strings
`artifacts/final.json`	host matrix, cookies, block state
`artifacts/bb-blocked.png`	the "Access denied" block page screenshot

9. Summary line

> *"BootBarn's bot protection is HUMAN Security (PerimeterX): a server-scored, block-page deployment. The client ships session/behavioral/device telemetry in the clear at the JSON boundary (then XOR-encodes it), fingerprints the GPU via an embedded WebGL shader, and the server decides 'automation' from UA + software-GL + launch signature + behavioral bundle — serving a captcha or the deny page. Client-readable, server-decided: the obfuscation (base91 strings, XOR wire) is a cost-raiser, not a boundary."*